

[LCS-202x] VHDL DPI/FFI based on GHDL's implementation of VHPIDIRECT

Unai Martinez-Corral*

Last updated on and generated on 2026/03/26

Contents

An autogenerated document	2
Contact	2
1 Introduction	3
2 Existing interfaces	4
2.1 VHPI	4
2.2 FLI	4
2.3 XSI	4
2.4 VPI	5
2.5 DPI	5
2.6 GHDL's VHPIDIRECT	5
2.6.1 Development	6
2.7 CXXRTL	7
3 Proposal: VFFI	7
3.1 Registration of foreign subprograms	7
3.2 Calling convention	8
3.3 Deferred definition of external subprograms	9
3.4 Function overloading	10
3.5 Interface modes: in, out, inout	10
3.6 Types	11
3.6.1 Enumeration	11
3.6.2 Integer	11

*Digital Electronics Design Group (ehu.eus/gded), Dept. Electronics Technology, University of the Basque Country (UPV/EHU), unai.martinezcorral@ehu.eus

3.6.3	Character	11
3.6.4	Real	12
3.6.5	Physycal	12
3.6.6	Time	12
3.6.7	Composite	13
3.7	A summary	16
4	Use cases	17
4.1	Executing arbitrary Python callbacks from VHDL	17
4.2	Raw simulation of hardware-software interactions on SoCs	17
4.3	Coarse grained mixed-language simulation	17
4.4	Coarse grained mixed-signal simulation	17
4.5	Using pipes or RPC libs to implement <i>virtual</i> peripherals	18
4.6	Implementing algebra algorithms in VHDL	18
4.7	Replacing functions in unmodified binaries, with behaviourally equivalent VHDL simulations/designs/models	18
4.8	Providing custom(izable) GUI interfaces to interact with simulations at runtime	18
5	Requirements	18
6	Existing C headers	19
	Glossary and acronyms	19
	References	21

List of Figures

An autogenerated document

This document (either HTML, PDF or EPUB) is autogenerated from the sources at the following GitHub repository: [umarcor/ghdl-cosim](#): [vhdl202x-lcd-dpi/vhdl202x-lcs-dpi](#). Hence, the content is expected to change, as fixes, enhancements or new sections are added. In HTML, at the top of each page, there are references to the history and sources. When discussing the content, please be aware that subtle differences might exist.

Contact

The VHDL Analysis and Standardization Group (VASG) is responsible for maintaining and extending the VHDL standard (IEEE 1076). The official website of the group is eda-twiki.org/cgi-bin/view.cgi/P1076. Interested users can subscribe to the reflector (mailing-list) and/or read the archives: grouper.ieee.org/groups/1076/email. Apart from that, in December 2019, VHDL libraries were published at opensource.ieee.org/vasg/Packages.

Although being a member of IEEE is not required, writing to the reflector or accessing ieee.org require users to register. In order to reach a wider audience, the VASG uses an organization in GitLab too: gitlab.com/IEEE-P1076.

Even though not directly related to the VASG, an organization named “Open Source VHDL Group” exists in GitHub: github.com/vhdl (find corresponding chat rooms: gitter.im/vhdl).

The content of this document is being discussed in the following thread of the VASG reflector: grouper.ieee.org/groups/1076/email/msg01286.html. There is also an issue in gitlab.com/IEEE-P1076/VHDL-Issues/-/issues/10. Moreover, there is a proposal in VASG’s TWiki site since 2011 named “Direct Programming Interface” (see eda-twiki.org/cgi-bin/view.cgi/P1076/DpiProposal). Nevertheless, interested users can participate through any of the forums referenced above.

1 Introduction

Traditionally, algorithms and systems are modeled using software languages. After developing refinements, defining the data flow and adjusting data types, specific parts of high-performance systems are typically accelerated by designing custom hardware IP (using hardware description languages). In the transition from a software algorithm to the hardware description of a circuit, expertises of hardware designers and software developers overlap. In such contexts, co-simulation, being able to co-execute HDL designs with *foreign* languages, is a very valuable feature. On the one hand, it allows software developers and verification engineers to test a given HDL *black box*, while using the languages and/or frameworks they are familiar with. On the other hand, it allows hardware designers to reuse in HDL the vast libraries available from software ecosystems (e.g. scipy.org). Moreover, most modern designs are complex SoCs where ad-hoc IP are placed along with general purpose (embedded) CPUs. Hence, testing co-execution of hardware and software is a must.

During the last two decades, multiple different and not exactly compatible interfaces/APIs have been proposed for co-simulation of HDL languages and foreign languages (mainly C/C++):

- VHDL Programming Interface (VHPI)
- Foreign Language Interface (FLI)
- Xilinx Simulator Interface (XSI)
- Verilog Procedural Interface (VPI)
- SystemVerilog Direct Programming Interface (DPI)

The underlying workflow in most of them is based on accessing the (V)HDL hierarchy of a given design at runtime, from a foreign language. Typically, callbacks are defined in C/C++ (as specified by the API). Some of the interfaces allow declaring additional models in C/C++/SystemC and having them registered for analysis/elaboration. Yet, the procedure is typically described from a software perspective, minimizing the usage of HDL language features, and keeping HDL sources unmodified. As far as the authors are aware, almost none of them is expected to support workflows where most of the co-simulation plumbing is handled in (V)HDL instead. By the same token, executing specific VHDL functions/procedures from the foreign language is typically not supported.

A subset of VHPI which is named VHPIDIRECT (thus, part of VHDL 2008) allows to define an **attribute** (FOREIGN) of subprograms in VHDL for importing functions written in foreign languages, and linking them during elaboration. It is a significant difference compared to *mainstream* workflows that this approach requires writing both VHDL and C/C++ (or other foreign language, such as Python). However, at the same time, metadata/context that needs to be defined when using an external language, is implicit in VHDL. Hence, VHPIDIRECT should feel more familiar to hardware developers, but it might be less intuitive to software developers. VHPIDIRECT is kind of similar to SystemVerilog’s DPI, since the main language is an HDL in both cases, and the main purpose is to interface C/C++. However, SystemVerilog is *per se* a language flavoured for software.

There is an existing proposal in the VASG since 2011, which has been maintained by Peter Flake, regarding extensions to the current FOREIGN attribute: [\[P1076/DpiProposal\] Direct Programming Interface](#). The list of individuals that have commented in that proposal is the following (in chronological order): John Shields, Tristan Gingold, David Koontz, Ryan Hinton and Peter Ladow. Furthermore, the following individuals did sign as supporters: David Smith, Brent Hayhoe and Radoslaw Nawrot.

The purpose of this document is to find common requirements and to conciliate the differences between VHPI, the existing DPI proposal and GHDL's VHPIDIRECT implementation. Opposed to existing VPI or VHPI, the purpose is to standardize an interface that feels natural to hardware designers (thus VHDL-centric), by allowing users/developers to reuse existing calling conventions in foreign languages, instead of requiring glue logic.

2 Existing interfaces

As mentioned in the introduction, in the last three decades multiple foreign/co-simulation interfaces for HDL have been proposed. In this section, the most relevant characteristics of each of them are explained. Note that many definitions and references can be found in the [glossary](#) at the end of this document. Hence, in this section, redundant references are avoided.

2.1 VHPI

VHDL Procedural Interface (VHPI) was incorporated in VHDL 2008. It is the only co-simulation interface defined for VHDL. It's described as "*application-programming interface to VHDL tools that allows programmatic access to a VHDL model during its analysis, elaboration, and execution*". Therefore, it is meant for implementing non-trivial so-called "*VHPI applications*". The programming model is based on a reference C header file with several layers of nested structs and callbacks of specific types for hooking each of the analysis, elaboration and execution stages.

Unfortunately, after more than a decade, the sparse or nonexistent support for the VHPI has been a problem. Compared to other interfaces, VHPI is so complicated and does not meet the needs of the average user. Therefore, it retarded interest by users, and that lack of interest has not pushed tool developers to go further. Anecdotically, GHDL supports VPI, but not VHPI, even though it's a VHDL only simulator.

It is to be noted that Verilog went down a similar path. Verilog has had the PLI since day one; it was fairly straightforward and easy to use. Then, VPI came and complicated things. In fact, VPI and VHPI are conceptually quite similar. Finally, Verilog pushed back and the DPI was created. Therefore, when designing a VFFI for VHDL, it'd be sensible to have Verilog's journey in mind.

Unlike Verilog users, who were accustomed to using a foreign interface and expected it to be available, VHDL have not had any foreign language interface traditionally. However, VHDL is known to be a more robust language, due to its strong typing system. Hence, there are VHDL specific cases that cannot be solved in the Verilog space. That's the critical mass for a VHDL FFI.

TODO: <https://forums.xilinx.com/t5/Simulation-and-Verification/Is-VHPI-support-planned-for-Vivado/td-p/562533>

2.2 FLI

TBW

2.3 XSI

TBW

2.4 VPI

TBW

2.5 DPI

TBW

2.6 GHDL's VHPIDIRECT

NOTE: As explained later, GHDL's VHPIDIRECT implementation is currently not compliant with the standard.

GHDL has supported VPI and VHPIDIRECT for a long time. GHDL is known to work on x86, ARM and/or PPC devices. Hence, it is relevant that GHDL's VHPIDIRECT is available in many more platforms than other interfaces. This is not a limitation imposed by the interfaces, but because of the lack of implementations. In April 2020, the documentation of GHDL regarding co-simulation with foreign languages was split to a separate repository: [ghdl/ghdl-cosim](https://github.com/ghdl/ghdl-cosim). The main motivation to do so was to better document and support the particular implementation of VHPIDIRECT in GHDL. Docs were updated and +20 examples were gathered/added (others are being worked on): [ghdl.github.io/ghdl-cosim](https://github.com/ghdl/ghdl-cosim). Those examples were contributed by Unai Martinez-Corral and user RocketRoss in GitHub. However, it is to be noted that most of them were based on existing code examples by others: Tristan Gingold, Yann Guidon, Rene Doss and Martin Strubel. See [ghdl/ghdl-cosim#1: Previous work and future ideas](https://github.com/ghdl/ghdl-cosim#1: Previous work and future ideas).

Before reworking `ghdl-cosim`, during 2019 Unai Martinez-Corral contributed [VUnit/cosim](https://github.com/vunit/vunit-cosim) (see also [vunit.github.io/cosim](https://github.com/vunit/vunit-cosim)). As explained in [VHDL Libraries > Data Types > External VHDL API](#), the purpose of `VUnit/cosim` is to expose some of VUnit's internal data types through GHDL's VHPIDIRECT (see [Bridges > VHPIDIRECT](#)). Since VUnit's Communication Library and Verification Components Library are based on those internal data types, the ultimate goal is to allow message passing between queues written in different languages (VHDL and Python). The development to extend the approach to the communication library is currently stalled because of details of VHPIDIRECT which are undefined in the standard (thus implementation-dependent) and which are not explicitly documented in GHDL. After multiple conversations with Tristan Gingold (the author of GHDL), Bradley Harden did a detailed analysis of the complexity to have 1-to-1 mapping between VHDL procedures/functions and C/C++ functions, in the context of cross-language queues. See [VUnit/vunit#603](#).

[hackfn/ghdlex](#) and `netpp`, by Martin Strubel, compose a framework on top of GHDL's VHPIDIRECT to allow exposing variables and other properties through the network. It provides *virtual* consoles, FIFOs, RAMs, etc. It handles several undocumented types of GHDL. Unfortunately, it is undocumented too.

GHDL is written in Ada. Ada and VHDL have multiple similarities. The compile and link models of Ada and C have multiple similarities. Overall, objects can be created in VHDL, Ada and/or C, and then linked together. Hence, the essence of how VHPIDIRECT is implemented in GHDL is that the runtime (GRT) exposes raw internal VHDL/Ada data structures/types. Unlike VPI or VHPI, there is no harness. However the following features are possible but undocumented:

- Passing complex data types (fat pointers).
- Executing VHDL functions/procedures from C.

The details about how VHDL types are mapped to C are explained in [Type declarations](#). The *fat pointers* mentioned there, which are used for unconstrained arrays and accesses to unconstrained arrays, are not documented. There is work in progress in [ghdl/ghdl-cosim#3](#) to provide a header file that helps with manipulating those fat pointers. The main purpose is to support multidimensional arrays with dimensions of type natural. That is, strings, vectors, matrices, cubes... Such header might be used by some of the projects

mentioned above (VUnit, ghdllex...). In fact, the purpose of this LCS is to include such header (an improved version of it) in the next revision of the standard. That is, to define the C representation of complex data types.

Note that in the documentation of ghdl-cosim ([ghdl.github.io/ghdl-cosim](https://github.com/ghdl/ghdl-cosim)) the term VHPIDIRECT is used loosely. Strictly, “*Wrapping a simulation (ghdl_main)*” and “*Linking object files*” are features provided by GHDL/GRT, which are NOT defined in the standard. In fact, those are not available with GHDL’s built-in in-memory backend (mcode). Those are only possible with LLVM or GCC. The same applies to section “*Dynamic loading*”. Loading foreign objects (from stdlib or shared libs) in a simulation is possible through VHPIDIRECT; however, generating a simulation model as a shared library is a feature provided by GHDL/GRT. The motivation to have VHPIDIRECT and GRT features mixed together is, precisely, this LCS.

Note also that option `-shared` was added to GHDL recently. Hence, there are still some issues when running simulations as shared libraries. See [ghdl/ghdl-cosim#15](https://github.com/ghdl/ghdl-cosim/issues/15).

See [Examples](#) for an step-by-step introduction to these co-simulation features. The first example is as simple as calling `rand` from `stdlib` in VHDL. The most complex one is to have an RGB image buffer drawn in an X11 window at runtime (during simulation). I think this [Virtual VGA screen](#) is very illustrative, because the UUT is unmodified. A VHDL (testbench) module is used to monitor HSYNC, VSYNC and RGB *signals* and place them in a 2D array of integers (the framebuffer). Then, the framebuffer is shared with C by reference (avoiding memory duplication). Actually, two different C implementations are shown: one saves snapshots in PNG images and builds an animated GIF, the other one uses the X11 window.

2.6.1 Development

Although undefined (or implementation dependent) in the standard, GHDL exposes arguments of [any](#) VHDL type through VHPIDIRECT. It is possible to inspect the structure of the fat pointers by using `gdb`:

- Comment all the foreign attribute lines in the source.
- Build the binary with GCC backend.
- Execute the binary in `gdb`:
 - `info functions <name>` to find the long name that the target function takes.
 - `p <long_name>` to print the prototype of the function.
 - `b <long_name>` to set a breakpoint.
 - `r` to run until the breakpoint.
 - `bt full` to see the arguments.

For example:

```
(gdb) bt full
#0  work__tb_c__ARCH__tb__print_string (PARAMS=0x557d3a627ef8, INSTANCE=0x557d3a627ee0) at tb_c.vhd:5
    FRAMEPTR = 0x557d3a627f20
    STATE = 21885
    T0_0 = {dim_1 = {left = 761498336, right = 32765, dir = (unknown: 24), length = 0}}
    T0_1 = 0x557d3a624710
    T0_2 = {BASE = 0x557d3a6246f0, BOUNDS = 0x557d3a4b82d8 <__gnat_malloc+24>}
    T0_3 = 0x557d3a4aef92 <<__ghdl_stack2_mark>+116>
    T0_4 = 32765
    T0_5 = 0x7ffd2d638ae0
    T0_6 = 0x557d3a624710
    _UI00000003 = {filename = 0x557d3a5486f0 <_UI00000000>, line = 8, col = 5}
```

2.7 CXXRTL

- <https://github.com/YosysHQ/yosys/tree/master/backends/cxxrtl>
- <https://tomverbeure.github.io/2020/08/08/CXXRTL-the-New-Yosys-Simulation-Backend.html>

3 Proposal: VFFI

In this section, VHPI definitions from the VHDL 2019 LRM are compared with GHDL's VHPIDIRECT implementation. Careful reading reveals that both sources solve complementary problems:

VHDL 2019 LRM, 17.1 General:

The VHDL Procedural Interface (VHPI) is an application-programming interface to VHDL tools that allows programmatic access to a VHDL model during its analysis, elaboration, and execution.

VHDL 2019 LRM, 17.2.1 General:

The VHPI consists of two aspects: - An information model that represents the topology and state of a VHDL model. - A number of functions that operate on the information model to access or affect the state of the VHDL model and that interact with tools during analysis, elaboration, or execution of the VHDL model.

<https://ghdl.github.io/ghdl-cosim/vhpirect>:

Interfacing with foreign languages through VHPIDIRECT is possible on any platform. You can reuse or define a subprogram in a foreign language (such as C or Ada) and import it into a VHDL design.

Hence, GHDL is currently not compliant with the VHPI, because it is using VHPIDIRECT's syntax to provide a feature which is out of scope of the wording in the standard. The general conception of VPI and/or VHPI is that those are interfaces to be used by software developers or, at least, people that don't want (or cannot) modify VHDL sources. As a result, the wording explains that the tool/simulator creates a model representing the VHDL sources/design, and a C/C++ interface is provided to interact with the design using software only. So, "*interact with tools during analysis, elaboration, or execution of the VHDL model*" in the LRM is describing a completely external interface that treats the VHDL model as an object from start to end. Conversely, in GHDL's VHPIDIRECT, analysis, elaboration and execution are done as any other regular VHDL design (the workflow is absolutely VHDL-centric), but specific (user pre-defined) foreign functions can be called from VHDL, to be executed in the same cycle. The conception of VHPIDIRECT as used in GHDL is just the opposite to VHPI: allowing a design written in VHDL to execute existing (preferredly unmodified) subprograms written in a foreign language.

3.1 Registration of foreign subprograms

From Annex B of the LRM, when registering foreign subprograms, the following structure must be used:

```
typedef struct vhpiforeignDataS {
    vhpiforeignKindT kind;
    char * libraryName;
    char * modelName;
    void (*elabf)(const struct vhpicbDataS *cb_data_p);
    void (*execf)(const struct vhpicbDataS *cb_data_p);
} vhpiforeignDataT;
```

And the definition of `vhpiCbDataS` is the following:

```
typedef struct vhpiCbDataS {
    int32_t reason;           /* callback reason */
    void (*cb_rtn) (const struct vhpiCbDataS *); /* call routine */
    vhpiHandleT obj;         /* trigger object */
    vhpiTimeT *time;         /* callback time */
    vhpiValueT *value;       /* trigger object value */
    void *user_data;         /* pointer to user data to be passed to the callback fun
} vhpiCbDataT;
```

As explained, those definitions describe a “C centric workflow”, where the main entrypoint to some application (tool) is written in C/C++. As a result, when defining callbacks and registering them in some pre-existing VHDL design (model), multiple context-related parameters need to be provided. Those are `libraryName`, `modelName`, `reason`, `object`, etc. On the other hand, when direct binding is used instead, `kind`, `libraryName` and `modelName` are implicit; `elabf` is optional; and `execf` suffices (see *VHDL 2019 LRM*, 20.2.4.3 *Standard direct binding*). GHDL supports direct binding only.

Due to the current wording in the LRM, it is unclear whether direct binding requires a function pointer to be declared, a function name, or any of them. In *VHDL 2019 LRM*, 20.2.4.3 *Standard direct binding*, `execution_function_name` is used, which is `execution_function_name ::= C_identifier`. In the structs above, `*elabf` and `*execf` are declared as pointers to functions. However, GHDL requires VHPIDIRECT strings to contain the name of functions, not pointers. This should be explicit. IMHO, both functions and pointers should be allowed.

3.2 Calling convention

According to the LRM, any foreign subprogram must be defined in C as receiving a single argument of type `vhpiCbDataS*` (see prototypes of `*elabf` and `*execf` above). Then, user data is passed in a field inside that struct. Conversely, GHDL allows binding foreign functions with multiple arguments of different types, and no intermediate struct is used. As a result, GHDL is not compliant with the standard; however, this is the main motivation for this proposal:

Currently the standard does not allow binding existing functions from std C lib (e.g. `rand`), or from any other existing shared library, without additional and verbose plumbing:

- Simulators need to implement providing all the fields/data in `vhpiCbDataS` and `vhpiForeignDataS`, even if `execf` and `user_data` are needed only.
- Even if the arguments/interfaces of the foreign existing functions match 1-to-1 with VHDL types (see below), users need to use two intermediate structs in C.

Overall, VHPI seems to fit the requirements of complex testing infrastructures in large design teams with different roles, some of them focused on verification. However, it feels too complex for the average RTL designer, and the learning curve is steep.

At the same time, VHPI seems to mimic the features and workflow of VPI, as if it was conceived just as the equivalent in a different language. As a matter of fact, GHDL implemented VPI first. When VHPI was published, developers didn’t see a motivation to duplicate the effort for no evident benefit. Still, the concept of “direct binding” was considered valuable, and foreign attribute `VHPIDIRECT` was implemented. Either because a misunderstanding, or because a draft was used, GHDL’s implementation of `VHPIDIRECT` resembles [Foreign Function Interface \(FFI\)](#) mechanisms in other languages.

The current proposal is to define an alternative calling convention in the standard, named VHDL Foreign Function Interface (VFFI), based on what GHDL provides already:


```
standard_direct_subprogram_binding ::=
VFFI object_library_specifier execution_specifier
```

Where `execution_specifier` is either `function_name` or `function_pointer_name`. The prototype of the function needs NOT to be provided in the foreign attribute declaration, but the number of arguments should match the number of arguments/interfaces in the VHDL subprogram, and the sizes of the types of each argument/interface must match (see below).

From a practical point of view, if a user wants to call a foreign C function from VHDL and pass a constrained array of integers, with VHPI iteration and calling multiple functions is required (see comment by PeterLadow in [DpiProposal#General_Comments](#)). With VFFI (GHDL's VHPIDIRECT), a pointer would be passed, which can be either indexed or dereferenced from C, and data can be manipulated in-place. For short code examples, see [ghdl-cosim: vhpirect/quickstart/random/tb.vhd](#) or [ghdl-cosim: vhpirect/quickstart/math/tb.vhd](#). Note that in those examples, writing C code is not required, because foreign functions are already defined in the std C lib or in a shared library.

3.3 Deferred definition of external subprograms

Given the following VHDL declaration:

```
procedure vplot (x: std_logic_vector(0 to 15)) is
begin report "VFFI vplot" severity failure; end;
attribute foreign of vplot : procedure is "VFFI plot";
```

either of the following should work:

```
void plot(int* x) {
    // do something
}
```

or

```
void (*plot)(int*);
```

Furthermore, since the prototype is known from the definition of the VHDL procedure, in the latter, the C source might be not required:

```
attribute foreign of plot: procedure is "VFFI _protoype_";
```

The purpose being to build VHDL sources without providing a body for the foreign subprogram. Then, from any foreign language:

1. Load the VHDL model.
2. Set define the `plot`.
3. Execute the simulation.

This approach is similar to the default workflow with VHPI as defined in the standard, albeit without intermediate structs.

3.4 Function overloading

VHDL subprograms might be overloaded, while C subprograms cannot. ~~It should be possible to have a couple of VHDL subprograms with the same name, each mapped to a different foreign subprogram.~~ Tristan pointed out that it is possible to provide a signature when defining an attribute. See VHDL 2019 LRM, 7.2 Attribute specification. Hence, although verbose, the following works:

```
function xyce_run(
  id      : string;
  reqTime : real
) return integer;
attribute foreign of xyce_run : function is "VHPIDIRECT xhdl_run";

function xyce_run(
  id      : string;
  reqTime : real;
  arrTime : real_vector;
  arrVolt : real_vector
) return integer;
attribute foreign of xyce_run[
  string,
  real,
  real_vector,
  real_vector
  return integer
] : function is "VHPIDIRECT xhdl_run_1D";

type arr2D_t is array (natural range <>, natural range <>) of real;
function xyce_run(
  id      : string;
  reqTime : real;
  arr2D    : arr2D_t
) return integer;
attribute foreign of xyce_run[
  string,
  real,
  arr2D_t
  return integer
] : function is "VHPIDIRECT xhdl_run_2D";
```

3.5 Interface modes: in, out, inout

For the `in` mode, non-composite types are passed by value, which correspond to the standard C/Ada mechanism. However, `out` and `inout` interfaces and complex types need special handling.

GHDL collects the interfaces of modes `out` or `inout` in a record/struct, which is passed by reference as the first argument to the foreign subprogram. The tool allocates the facade of the struct, but not the buffers that contain the actual data. Hence, foreign functions need to allocate and assign data to the placeholders. As a consequence, it is not suggested to use `out` and/or `inout` modes in foreign subprograms with GHDL, since they are not portable.

The proposed standardization is NOT to gather the “special” interfaces in any record/struct. Instead, the position of the arguments should be preserved, and the facade for each “special” interface should be provided in the expected position. Naturally, modes `out` or `inout` will not be compatible with pre-existing C libraries.

However, because it is a direct approach, the glue logic is significantly less complex than using a full-featured VHPI API.

3.6 Types

On the one hand, the representation of the types of the interfaces between VHDL and foreign languages when using GHDL's VHPIDIRECT are defined in [Type declarations > Restrictions on type declarations](#). On the other hand, in the VHDL 2019 LRM the same content is defined in chapter/clause “22. VHPI value access and update” and in “Annex B (normative) VHPI header file”. In this section, both sources are compared.

3.6.1 Enumeration

VHDL 2019 LRM, 22.2.2 vhpEnumT and vhpSmallEnumT:

A value of type `vhpEnumT` or `vhpSmallEnumT` represents a value of a VHDL enumeration type. A value of type `vhpEnumT` shall be represented with at least 32 bits, and a value of type `vhpSmallEnumT` shall be represented with at least 8 bits. The value represented by a given value of either type is an enumeration value whose position number is the given value, interpreted as an unsigned binary number.

GHDL

8 bit word, or, if the number of literals is greater than 256, by a 32 bit word. There is no corresponding C type, since arguments are not promoted.

Both approaches are similar. However, in GHDL, the tool implicitly decides whether `vhpEnumT` or `vhpSmallEnumT` are to be used, depending on the size of the enumeration in VHDL. In VHPI, it feels that users can get a value of an small enumeration as a 32 bit word. That is not supported in GHDL.

3.6.2 Integer

VHDL 2019 LRM, 22.2.3 vhpIntT and vhpLongIntT:

A value of type `vhpIntT` or `vhpLongIntT` represents a value of a VHDL integer type. A value of type `vhpIntT` shall be represented with at least 32 bits, and a value of type `vhpLongIntT` shall be represented with at least 64 bits. The value represented by a given value of either type is the given value, interpreted as a signed twos-complement binary number.

GHDL

32 bit word. Generally, `int` for C or `Integer` for Ada.

GHDL does not support integers of 64 bits yet. Still, the implementation is likely to be similar to the enumerations (see above).

3.6.3 Character

VHDL 2019 LRM, 22.2.4 vhpCharT:

A value of type `vhpCharT` represents a value of a VHDL character type. The value shall be represented with at least 8 bits. The value represented by a given value of type `vhpCharT` is a character value whose position number is the given value, interpreted as an unsigned binary number.

In GHDL, characters are considered enumerations with 256 literals (8 bits). Hence, both approaches are equivalent.

3.6.4 Real

VHDL 2019 LRM, 22.2.5 vhpiRealT:

A value of type `vhpiRealT` represents a value of a VHDL floating-point type. The value shall be represented with at least 64 bits. The value represented by a given value of type `vhpiRealT` is the given value, interpreted according to the chosen representation for floating-point types (see 5.2.5.1).

GHDL

64 bit floating point word. Generally, double for C or Long_Float for Ada.

Both approaches are equivalent.

3.6.5 Physycal

VHDL 2019 LRM, 22.2.6 vhpiPhysT and vhpiSmallPhysT:

A value of type `vhpiPhysT` is called a physical structure and represents a value of a physical type. The position number of a physical structure is the signed integer represented by the concatenation of the high and low members of the physical structure to form a 64-bit twos-complement binary number, with the high member as the most significant part and the low member as the least significant part.

A value of type `vhpiSmallPhysT` also represents a value of a physical type. The value shall be represented with at least 32 bits. The position number of the value of type `vhpiSmallPhysT` is the value interpreted as a signed twos-complement binary number.

If a physical structure or value of type `vhpiSmallPhysT` occurs as part of a value structure or as an element of an array pointed to by a value structure, its position number determines the value represented by the value structure or value of type `vhpiSmallPhysT`, as described in 22.2.8. Otherwise, the physical structure or value of type `vhpiSmallPhysT` represents a value of a physical type. The value is the product of the position number of the physical structure or value of type `vhpiSmallPhysT` and a unit determined from the context in which the physical structure or value of type `vhpiSmallPhysT` occurs.

GHDL

64 bit word. Generally, long long for C or Long_Long_Integer for Ada.

Both approaches are similar. However, GHDL does not support small physical types.

3.6.6 Time

VHDL 2019 LRM, 22.2.7 vhpiTimeT:

A value of type `vhpiTimeT` is called time structure and represents a time value. The position number of a time structure is the signed integer represented by the concatenation of the high and low members of the time structure to form a 64-bit twos-complement binary number, with the high member as the most significant part and the low member as the least significant part.

If a time structure occurs as part of a value structure or as an element of an array pointed to by a value structure, its position number determines the value represented by the value structure, as described in 22.2.8. Otherwise, the time structure represents a value of type `TIME` defined in package `STANDARD`. The value is the product of the position number of the time structure and the resolution limit of the tool.

NOTE—A VHPI program can determine the resolution limit with the function call `vhpi_get_phys(vhpiResolutionLimit, NULL)`.

In GHDL, although undocumented yet, `int64_t` is used.

3.6.7 Composite

VHDL 2019 LRM, 22.2.8 vhpValueT:

A value of type `vhpValueT` is called a value structure and represents a scalar value, a one-dimensional array of scalar values, or a value of any type represented in an implementation-defined internal representation.

The `format` member of a value structure specifies the format of the value structure, that is, a value of type `vhpFormatT` that determines how the value is represented. The value member of the value structure is a union that contains the value in the appropriate representation. The following formats are specified by this standard:

- `vhpBinStrVal`, `vhpOctStrVal`, `vhpDecStrVal`, `vhpHexStrVal`
- `vhpEnumVal`, `vhpSmallEnumVal`, `vhpIntVal`, `vhpLongIntVal`, `vhpLogicVal`, `vhpRealVal`, `vhpStrVal`, `vhpCharVal`, `vhpTimeVal`, `vhpPhysVal`, `vhpSmallPhysVal`, `vhpObjTypeVal`, `vhpPtrVal`
- `vhpEnumVecVal`, `vhpSmallEnumVecVal`, `vhpIntVecVal`, `vhpLongIntVecVal`, `vhpLogicVecVal`, `vhpRealVecVal`, `vhpTimeVecVal`, `vhpPhysVecVal`, `vhpSmallPhysVecVal`, `vhpPtrVecVal`, `vhpRawDataVal`

An implementation may specify further formats and the way in which values are represented for those formats.

If a value structure is used by a VHPI program as an argument to the `vhp_get_value` function and the format of the value structure specifies an array, string, or internal representation, the VHPI program shall set the `bufSize` member of the value structure to the number of bytes of storage allocated by the VHPI program for storage of the value (see 23.19).

If the format of a value structure used to represent a value specifies an array or string representation, the `numElems` member of the value structure specifies the number of elements in the array or string representation of the value represented by the value structure. If the value is represented as a string, the number of elements excludes the null termination character of the string.

If the format of a value structure used to represent a value specifies a physical type or time type representation, the `unit` member of the value structure specifies a unit of the physical or time type. The position number of the value represented by the value structure is the product of the position number of the unit and the position number of the physical or time structure or value of type `vhpSmallPhysT` used to represent the value.

NOTE 1—A VHPI program that allocates buffer storage for a string to be written by a call to the `vhp_get_value` function should allow storage for the null termination character. The value written to the `bufSize` member of the value structure should be at least one more than the length of the string.

NOTE 2—The `vhpRawDataVal` format allows a VHPI program to read the value of an object without requiring the tool to reformat the value. An implementation may allow a VHPI program to read the value of an object in its internal representation and subsequently to set the value of an object of the same type using the value, thus avoiding the performance impact of reformatting.

```
typedef struct vhpValueS {
    vhpFormatT format;
    size_t bufSize;
    int32_t numElems;
    vhpPhysT unit;
```

```

union {
    vhpiEnumT      enumv,      *enumvs;
    vhpiSmallEnumT smallenumv, *smallenumvs;
    vhpiIntT       intg,       *intgs;
    vhpiLongIntT   longintg,   *longintgs;
    vhpiRealT      real,       *reals;
    vhpiSmallPhysT smallphys,  *smallphyss;
    vhpiPhysT      phys,       *physs;
    vhpiTimeT      time,       *times;
    vhpiCharT      ch,         *str;
    void           *ptr,       **ptrs;
} value;
} vhpiValueT;

```

Note that the types in the `value` union are the ones listed in previous subsections, plus `void`.

GHDL

Composite types:

- Records are represented like a C structure and are passed by reference to subprograms.
- Arrays with static bounds are represented like a C array, whose length is the number of elements, and are passed by reference to subprograms.
- Unconstrained arrays are represented by a fat pointer. It is not suggested to use unconstrained arrays in foreign subprograms.
- Accesses to an unconstrained array are fat pointers. Other accesses correspond to an address/pointer and are passed to a subprogram like other non-composite types.
- Files are represented by a 32 bit word, which corresponds to an index in a table.

Once again, the most significant difference is that VHPI requires an intermediate struct to be used, and field `format` defines the type of the `value`. Conversely, in GHDL the format is implicit (because it is a direct interface), and only the value is passed.

Furthermore, in VHPI one-dimensional arrays are defined only. Supporting other types through an implementation-defined internal representation is allowed, but undefined. This is reflected in the existence of fields `bufSize` and `numElems`, which can contain the size of a single dimension. GHDL supports either constrained or unconstrained multi-dimensional arrays, but uses different approaches:

- Constrained arrays are passed by reference, and no information about the size/length is given explicitly. This is true for multi-dimensional arrays too. Those are ordered in row-major or column-major order, depending on the underlying compiler/architecture.
 - Naturally, due to the strong typing in VHDL, it is up to the users to pass sizes as additional arguments (see [ghdl-cosim: Examples > Arrays > Constrained/bounded integer arrays > Sized in VHDL](#) and github.com/ghdl/ghdl-cosim/tree/master/vhpidirect/arrays/intvector/vhdsized). This is flexible enough to interact with predefined functions that require a pointer and several constants, but also with functions that expect those to be wrapped in some struct/record.
 - Note that the actual size of the buffer is implicit, because both VHDL and C are aware of the dimensions and the type of the values in the array. Furthermore, the type of the values can be any simple or composite/nested type. E.g., a constrained multi-dimensional array of records containing constrained one-dimensional arrays and integer fields/elements.
- Unconstrained arrays are passed as fat pointers. By the same token, unconstrained arrays in nested structures (records/arrays) are fat pointers too. Strictly, it might be considered that this is covered by the standard as `vhpiRawDataVal`, which is explained as “*an implementation may allow a VHPI program*

to read the value of an object in its internal representation”. However, the purpose of `vhpiRawDataVal` is “to set the value of an object of the same type using the value, thus avoiding the performance impact of reformatting”. In the context of GHDL and of this proposal, the purpose is to actually use/modify the content. Hence, it needs to be defined.

The proposal is to define fat pointers as follows:

```
typedef struct {
    void* data;
    int dim;
    void* bounds;
} fat_pointer_t;
```

The type of `data` would be defined by the type of the values in the array, according to the previous discussions. `dim` would tell the number of dimensions of the (multi-dimensional) array. Last, `bounds` would be an array of a type defined by the type of index used in VHDL. This is the major difference compared to `vhpiValueS`: allowing any type of index, instead of being constrained to `size_t bufSize` and `int32_t numElems`.

The most common index type is `integer` or `natural`, which translates to:

```
typedef struct {
    int32_t left;
    int32_t right;
    int32_t dir;
} range_t;
```

For example, type `arr_t` is array (natural range <>, natural range <>, natural range <>) of `std_logic_vector(7 downto 0)`; would be represented as follows:

```
typedef struct {
    char* data;
    int dim;
    range_t* bounds;
} fat_pointer_natural_multidim_array_of_stdv_t;
```

```
{
    data: char*,
    dim: 3,
    [
        {
            left: int,
            right: int,
            dir: int,
        },
        {
            left: int,
            right: int,
            dir: int,
        },
        {
            left: int,
            right: int,
            dir: int,
        }
    ]
}
```

```

    }
  ]
}

```

In fact, these structs might be proposed as an update to `vhpiValueS`:

```

typedef struct vhpiValueS {
    vhpiFormatT format;
    int dim;
    void* bounds;
    size_t bufSize;
    vhpiPhysT unit;
    union {
        vhpiEnumT      enumv,      *enumvs;
        vhpiSmallEnumT smallenumv, *smallenumvs;
        vhpiIntT        intg,       *intgs;
        vhpiLongIntT    longintg,   *longintgs;
        vhpiRealT       real,       *reals;
        vhpiSmallPhysT  smallphys,  *smallphyss;
        vhpiPhysT       phys,       *physs;
        vhpiTimeT       time,       *times;
        vhpiCharT       ch,         *str;
        void            *ptr,       **ptrs;
    } value;
} vhpiValueT;

```

However, this would be a breaking change (as it would effectively deprecate `vhpi*VecVal` types) that the committee might not want to introduce. Instead, the current proposal is to add a `vffi_user.h` file in the standard, which complements `vhpi_user.h`. All common types can be reused, and fat pointers for unconstrained, accesses to an unconstrained arrays, etc. would be defined in the new header file; along with helper function for accessing the most common multi-dimensional arrays.

There is a demo in `ghdl-cosim`, which showcases how `vffi_user.h` would look like, and how helper functions can be used in different use cases. See [ghdl/ghdl-cosim: vhpirect](#). Nevertheless, note that the demo is based on the current implementation of VHPIDIRECT in GHDL. It is expected to change after the outcome of this proposal. Instead of exposing the internal representation of fat pointers, an intermediate abstraction layer will be used in GHDL for complying with the standard.

3.7 A summary

Providing a direct interface that allows VHDL to reuse the FFI mechanism already available in other languages is a very useful enhancement for allowing reuse of existing advanced libraries in C, Ada, Python, Ruby, Julia, Rust, etc. Many of those languages are used in data science, where large amounts of multi-dimensional data are processed. At the same time, precisely due to the high dimensionality of data sets, usage of hardware design languages/generators and FPGAs accelerators is growing. In this context, it feels sensible to standardize not only scalar types, but also the types used for matrices, cubes and other multi-dimensional structures. That would allow known projects (such as numpy or tensorflow) to provide “VHDL bindings”, by writing VHDL and C sources once only.

Terms “GHDL’s VHPIDIRECT”, VFFI, VDPI, DPI, etc. in this document, all refer to the same concept: a new calling convention and an enhancement to the features of attribute `FOREIGN`.

4 Use cases

Although non standardized, GHDL has provided a FFI interface for more than a decade already. As a result, dozens of working examples exist, which showcase most of the features in this proposal. The following is a non-exhaustive list of different use cases:

4.1 Executing arbitrary Python callbacks from VHDL

By using `ctypes`, Python can be effectively seen as C from VHDL's/GHDL's perspective. I.e., it is possible to execute any function in any existing Python library as any other VHDL procedure/function (with constraints related to types). See, for example, `pycb`, where `matplotlib.pyplot` is used to draw an `x,y` graph of constrained one-dimensional arrays of integers from VHDL.

Using GHDL, it is currently not possible to “defer” the definition as proposed in [Deferred definition of external subprograms](#). At the same time, Python's `ctypes` does currently not support setting/overwriting a function pointer in a shared library. A helper C function needs to be used. Find a discussion about these limitations in [ghdl/ghdl#1398](#).

4.2 Raw simulation of hardware-software interactions on SoCs

The main purpose being to avoid the complexity of interconnects (i.e. delaying those implementation details to later in the design process). Verification Components, such as VUnit's (I'd expect OSVVM's [\[5\]](#) to work too) are off-the-shelf models that allow to interact with UUT's including *complex* bus interfaces following a software approach (testbench). That is, behavioural models for the interconnects are already available. Vivado HLS projects exported to VHDL have been successfully co-simulated with GHDL + VUnit only.

4.3 Coarse grained mixed-language simulation

At FOSDEM 2016, Tristan Gingold provided a demo about how to co-simulate VHDL and SystemC. Sources are available at [ghdl/ghdl-cosim/tree/master/systemc](#).

[Verilator](#) [\[8\]](#) is tool that allows generating cycle-accurate behavioural C++ models from synthesizable Verilog sources. In [ghdl/ghdl#1335](#), Benjamin Herrenschmidt reported combining microwatt (VHDL) with Litedram (Verilog) by using Verilator and GHDL's VHPIDIRECT.

4.4 Coarse grained mixed-signal simulation

[Xyce](#) ([github.com/Xyce/Xyce](#)) is an SPICE-compatible high-performance analog circuit simulator developer in Sandia National Laboratories (SNL), which uses solvers from Trilinos. [Trilinos](#) ([github.com/trilinos/Trilinos](#)), is a library focused on linear algebra techniques. In the end of 2018, several application notes regarding co-simulation were published at [Xyce: Documentation & Tutorials](#):

- Coupled Simulation with the Xyce General External Interface [\[11\]](#)
- Mixed Signal Simulation with Xyce¹ [\[6\]](#)
- Digital/Analog Cosimulation using CocoTB and Xyce [\[7\]](#)

In December 2019, U. Martinez-Corral rewrote the Xyce examples using GHDL's VHPIDIRECT, instead of cocotb and VPI. See [ghdl/ghdl#1052](#). In the context of this proposal, in September 2020 the examples were modified for using `vffi_user.h` (see [umarcor/ghdl-cosim: vhpirect/vffi_user/xyce](#)):

¹Note that later versions of the document exist. For example, [June 2019](#) and [June 2020](#)

- `runACircuit` shows how to get a null terminated C string from the fat pointer of an unconstrained string.
- `runACircuitInSteps` shows how to pass and use unconstrained `real_vector`.
- `runWithDACs` shows how to pass and use unconstrained multidimensional arrays of `real`.

4.5 Using pipes or RPC libs to implement *virtual* peripherals

FIFOs (queues), RAMs, screens, motors.... See [hackfin/ghdlex](#), [dbhi/gRPC](#) and [dbhi/vboard](#).

This also enables splitting large simulations in multiple low-cost devices which include the same ARM cores as other high-end SoC boards. Since GHDL works on ARM, hardware-software co-simulations can be executed *natively* (i.e. without using QEMU to emulate the software). See [dbhi.github.io/pdf/FPGA2020_poster.pdf](#).

4.6 Implementing algebra algorithms in VHDL

Passing matrices to/from Octave/Matlab/Scilab as pointers (by reference). This allows using VHDL 2008's `fixed_point` and `floating_point` packages as an alternative to “Fixed-point modeling” toolboxes. See [Array and AXI4 Stream Verification Components](#), [dbhi/fpconv](#) and [umarcor/MSEA/tree/master/octave](#).

From an opposite point of view, Octave/Matlab/Scilab can be used to implement models of *plants* in control applications, while designing all the system in VHDL. This is an alternative to Xilinx's integration with Simulink, where the system is designed in Simulink and VHDL source can be included as black boxes.

4.7 Replacing functions in unmodified binaries, with behaviourally equivalent VHDL simulations/designs/models

This allows to evaluate hardware accelerator designs without modifying the software to be accelerated. See [dbhi.github.io](#).

4.8 Providing custom(izable) GUI interfaces to interact with simulations at runtime

For example, in [VUnit/vunit#568](#) a Flask (Python) web server and a Vue (JavaScript) web frontend are used to visualize the arrays/buffers from the VHDL design. The last example is specially illustrative because it shows multiple internal buffers in an image processing pipeline.

5 Requirements

- Attribute `FOREIGN` should allow to define more than one foreign “language” or “implementation”. For example, it should be possible to use the same VHDL source to co-simulate with either GHDL's `VHPI` or Mentor Graphics' `FLI`. Currently, the format of the attribute string is not standardized, and the implementation of those tools is incompatible. Apart from that, VHDL sources are fully compatible.
- Currently, providing a (dummy) body for foreign functions is required, which cannot be used, because the function is precisely declared as foreign. It would be nice for the body not to be required. Alternatively, the body might be used as a fallback in case a foreign implementation is not found. Currently, it is required but unused, even as a fallback.

- Currently, if a procedure/function is decorated with the foreign attribute, apart from a matching body in VHDL, the foreign subprogram must be defined (linked) for elaboration to succeed. This is required even when it is not used in the design. In contexts where conditional inclusion of VHPIDIRECT features is required, it is suggested to provide alternative (dummy) sources for either VHDL packages or C sources. Can this be solved/simplified through conditional compilation (VHDL 2019)?
- Attribute FOREIGN should allow to bind a either foreign function name or a pointer/reference. This is useful for defining foreign functions at runtime through interpreted languages such as Python. Currently, function pointers need to be dereferenced for VHPIDIRECT to call them.
- It is currently possible to define foreign procedures or functions. However, the format of the complex data types that can be passed as arguments, is not defined. Assuming that most simulators are written in a language “similar enough to C”, passing simple data types is straightforward. E.g., **integers** are **int32_t** or **int64_t** and **reals** are **doubles**. Nevertheless, complex data types -such as (multidimensional) (un)constrained arrays, nested records, etc.- can be implemented very differently. More precisely, since data types in VHDL are strongly typed, many of them cannot be passed as a single reference; instead, fat pointers need to be used. The format of such fat pointers should be standardized.
 - By the same token, VHDL procedures/functions can have arguments of mode **in**, **out** or **inout**, whereas in C functions are allowed to return a single argument. Hence, this is currently implementation-dependent.
- It is possible to import a C subprogram in VHDL. However, it is currently undefined how to call a VHDL function/procedure from C. For C to find it, apart from making the symbol visible, it needs to have a known name. Such name is currently not standardized.
 - The same issues about non-standardized argument type conversion applies.
- Memory management needs to be defined. When pointers are passed between VHDL and the foreign language, users might attempt to allocate some variable/array in one language and deallocate/free the memory in the other language. That might or might not be possible and/or supported. Currently, it is undefined.
- ~~Is it possible to support varargs so that calling **printf** from VHDL is a “built-in” feature in the language?~~

6 Existing C headers

In this proposal, `vhpi_user.h` from opensource.ieee.org/vasg/Packages is used. However, inspecting other open source projects reveals that different versions of headers for VHPI and VPI are used.

- Verilator: [verilator/verilator@master](https://github.com/verilator/verilator@master): `include/vltstd`
- cocotb: [cocotb/cocotb@master](https://github.com/cocotb/cocotb@master): `cocotb/share/include`
- GHDL:
 - [ghdl/ghdl@master](https://github.com/ghdl/ghdl@master): `src/grt/vpi_user.h`
 - [ghdl/ghdl@master](https://github.com/ghdl/ghdl@master): `src/grt/vpi_thunk.h`
 - [ghdl/ghdl@master](https://github.com/ghdl/ghdl@master): `src/grt/vpi_thunk.c`

Glossary and acronyms

- **API**: Application Programming Interface (see [Wikipedia: Application programming interface](https://en.wikipedia.org/wiki/Application_programming_interface)).
- **ABI**: Application Binary Interface (see [Wikipedia: Application binary interface](https://en.wikipedia.org/wiki/Application_binary_interface)).
- **BFM**: Bus Functional Model.

- **cocotb**: a coroutine based cosimulation library for writing VHDL and Verilog testbenches in Python [4].
- **DPI**: Direct Programming Interface. SystemVerilog DPI is an interface which can be used to interface SystemVerilog with foreign languages (see [Wikipedia: SystemVerilog DPI](#)).
- **DUT**: Design Under Test.
- **FFI**: Foreign Function Interface, a mechanism by which a program written in one programming language can call routines or make use of services written in another (see [Wikipedia: Foreign function interface](#)).
- **FIFO**: First In First Out, a method for organising the manipulation of a data structure where the oldest (first) entry, or *head* of the queue, is processed first (see [Wikipedia: FIFO \(computing and electronics\)](#)).
- **FLI**: Foreign Language Interface. Mentor Graphics' [ModelSim/Questa](#) tools implement an interface named *VHDL FLI*: “*FLI routines are C programming language functions that provide procedural access to information within Model Technology's HDL simulator, vsim*”.
- **FOSS/FLOSS**: Free/Libre and Open Source Software.
- **GCC**: the GNU Compiler Collection [9].
- **GDB**: the GNU Project Debugger [10].
- **GHDL**: open-source analyzer, compiler, simulator and (experimental) synthesizer for VHDL [3].
- **GRT**: GHDL's RunTime library.
- **HDL**: Hardware Description Language.
- **HLS**: High Level Synthesis.
- **IEEE**: Institute of Electrical and Electronics Engineers (see [ieee.org](#)).
- **IR**: Intermediate Representation.
- **LCS**: Language Change Specification.
- **LLVM**: a compiler infrastructure designed around a language-independent IR [1].
- **OSVVM**: Open Source VHDL Verification Methodology [5].
- **PLI**: Program Language Interface. PLI was superseded by VPI in IEEE Std 1364-2005, sometimes referred to as *PLI 2*.
- **PPC**: commonly used to refer to PowerPC and/or Power ISA processors.
- **PSL**: Property Specification Language, IEEE Std 1850, incorporated in IEEE Std 1076-2008 (see [Wikipedia: Property Specification Language](#)).
- **QEMU**: a generic and open source machine emulator and virtualizer.
- **RPC**: Remote Procedure Call.
- **RTL**: Register-Transfer Level.
- **UUT**: Unit Under Test.
- **VASG**: VHDL Analysis and Standardization Group (see [eda-twiki.org/cgi-bin/view.cgi/P1076](#)).
- **VC**: Verification Component.

- **Verilator**: FLOSS tool for converting Verilog to a cycle-accurate behaviour model in C++ or SystemC [8].
- **Verilog**: a HDL, IEEE Std 1364. In 2009, it was merged into the SystemVerilog standard creating IEEE Std 1800 (see [Wikipedia: Verilog](#)).
- **VHDL**: VHSIC-HDL, Very High Speed Integrated Circuit Hardware Description Language, IEEE Std 1076 (see [Wikipedia: VHDL](#)).
 - Revisions: **1987**, **1993**, **2000**, **2002** (IEC 61691-1-1:2004), **2008** (IEC 61691-1-1:2011), **2019**.
 - Related standards:
 - * 1076.1 VHDL Analog and Mixed-Signal (VHDL-AMS)
 - * 1076.1.1 VHDL-AMS Standard Packages (stdpkgs)
 - * 1076.2 VHDL Math Package
 - * 1076.3 VHDL Synthesis Package (vhdsynth) (numeric_std)
 - * 1076.3 VHDL Synthesis Package – Floating Point (fphdl)
 - * 1076.4 Timing (VHDL Initiative Towards ASIC Libraries: vital)
 - * 1076.6 VHDL Synthesis Interoperability (withdrawn in 2010)[11]
 - * 1164 VHDL Multivalued Logic (std_logic_1164) Packages
- **VHPI**: VHDL Procedural Interface, incorporated in IEEE Std 1076-2008.
- **VPI**: Verilog Procedural Interface, incorporated in IEEE Std 1364-2005 (see [Wikipedia: Verilog Procedural Interface](#)).
- **VUnit**: an open source unit testing framework for VHDL/SystemVerilog [2].
- **XSI**: Xilinx Simulator Interface, “a C/C++ application programming interface (API) to the Xilinx Vivado Simulator (xsim) that enables a C/C++ program to serve as the test bench for a HDL design” [12].

References

- [1] Vikram Adve, Chris Lattner, and LLVM Developer Group. *LLVM Project, a collection of modular and reusable compiler and toolchain technologies*. 2003. URL: <https://www.llvm.org/>.
- [2] Lars Asplund, Olof Kraigher, and contributors. *VUnit: a unit testing framework for VHDL/SystemVerilog*. Sept. 2014. URL: <http://vunit.github.io/>.
- [3] Tristan Olof Gingold and contributors. *GHDL: open-source analyzer, compiler, simulator and (experimental) synthesizer for VHDL*. 2003. URL: <http://ghdl.free.fr/>.
- [4] Chris Higgs, Stuart Hodgson, and contributors. *Coroutine Cosimulation TestBench (cocotb)*. June 2013. URL: <https://github.com/cocotb/cocotb>.
- [5] Jim Lewis and contributors. *Open Source VHDL Verification Methodology (OSVVM)*. May 2013. URL: <https://osvvm.org/>.
- [6] Peter E. Sholander and Richard Schiek. “Application Note: Mixed Signal Simulation with Xyce”. In: (Nov. 2018). DOI: [10.2172/1483152](https://doi.org/10.2172/1483152). URL: <https://www.osti.gov/biblio/1483152>.
- [7] Andrew Michael Smith et al. *Digital/Analog Cosimulation using CocoTB and Xyce*. Dec. 2018. DOI: [10.2172/1488489](https://doi.org/10.2172/1488489). URL: <https://www.osti.gov/biblio/1488489>.
- [8] Wilson Snyder and contributors. *Verilator, FOSS tool which converts Verilog to a cycle-accurate behavioral model in C++ or SystemC*. 2003. URL: <https://www.veripool.org/wiki/verilator>.
- [9] Richard Stallman and contributors. *GCC, the GNU Compiler Collection*. May 1987. URL: <http://gcc.gnu.org/>.

- [10] Richard Stallman and GNU Project. *GDB: The GNU Project Debugger*. 1986. URL: <https://www.gnu.org/software/gdb/>.
- [11] Thomas V. Russo and Russell Hoope. *Coupled Simulation with the Xyce General External Interface*. Dec. 2018. DOI: [10.2172/1488489](https://doi.org/10.2172/1488489). URL: <https://www.osti.gov/biblio/1488489>.
- [12] Xilinx. *UG900 (v2018.3) Vivado Design Suite User Guide, Logic Simulation*. 2018. URL: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2018_3/ug900-vivado-logic-simulation.pdf.